C = Programming → Source Code
→ flow chart
→ Algorithm

Inventor :- Dennis Ritchie

Case sensitive programming

file extention '.c'

- # include <stdio.h> → Preprocessor Directive

■ Preprocessor Directive :- It is used to call the header file before processing.

■ include :- It is a key word which is used to include the header file within the programme before the prossing has been started.

<> :- header file sign.

■ <stdio.h> :- 'Standard Input Output Header file' It is used to import the system defined function in our programming code.

① Printf( )

② Scanf( )

■ Printf( ) :- It is a system defined fanction which is present within <stdio.h>. It is used to print or display anything on the screen.

■ Scanf( ) :- It is a also system defined function which is present within <stdio.h>, is also used to take input from user.

include <conio.h>

<conio.h> :- 'Console input output'. It is a header file which consists of getch() which is used to stay the output on the screen, until any key is processed output will not closed.

* console mean black screen.

Datatype

- Integer :- Keyword :- int (It only stores number)
  eg:- a = '65'
  denoted by :- %d size : 2-4 byte
- Float :- Keyword :- float (It stores fraction numbers) eg:- a = '65.001016'
  capacity :- 6 decimal place denoted by :- %f size: 8 byte
- Character :- Keyword → char (It only stores single letters) eg:- 'c'
  denoted by : %c size = 1 byte
- Double :- Keyword → double (It also stores function number) eg:- '5.776894 32 145781'
  capacity: 14 decimal denoted :- %lf size: 16 byte.
- String :- Keyword → string (It contains multiple words) (It also stores special characters and space.)
  (char(array))
  denoted by = %s size : NA.

(ES-CS-1900) ES-CS19,

```
void main()
{
 ---
 ---
 ---
 ---
}
```

BODY OF MAIN()

Statment

void main(); It is a system defind function, within the body section of main() we have to write the programme to get the desired output. because operating system can only identify main().

';' = Terminating operators:- It is a operator which is used to give instruction to the computer. that the statement has been terminated or on end.

```
Printf("Subho");
Scanf("%d", &a)
```

[& : ampersand/address operater]

Printf(); It is used to display anything on the screen we have to write the word or sentence within the double quetation mark.

& :- It is an address operator which is used to store the varibles valus within the bod variables value within the RAM memory.

Datatype :- "%d" or any datatype is wanted.

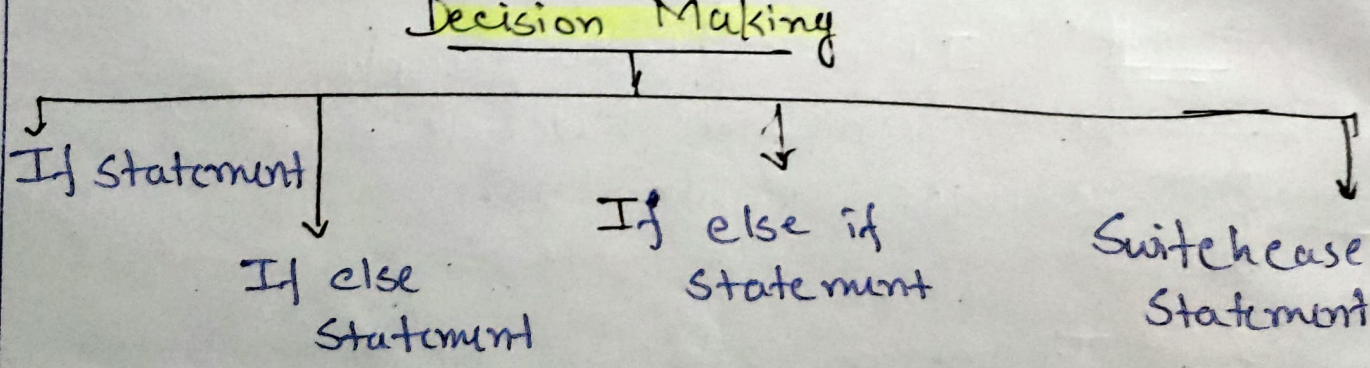
"\n" It is an escape sequence which is used to put the cursor into the next line.

Theory – if else

/ → (Divide) :- It is a divide operator which is used to find out the quotient of any number.

% → Modulus :- It is a modulus operator which is used to find out the reminder of any number.

Decision Making Statement :-



If Statement :- It is a desition muking Statement which is used when we want to chuck any speeific condition. if the condition is satisfied or true, then the remaining portion with in the if statement will be executed. If the condition will unsatisfied or false, nothing will be display on the seneen.

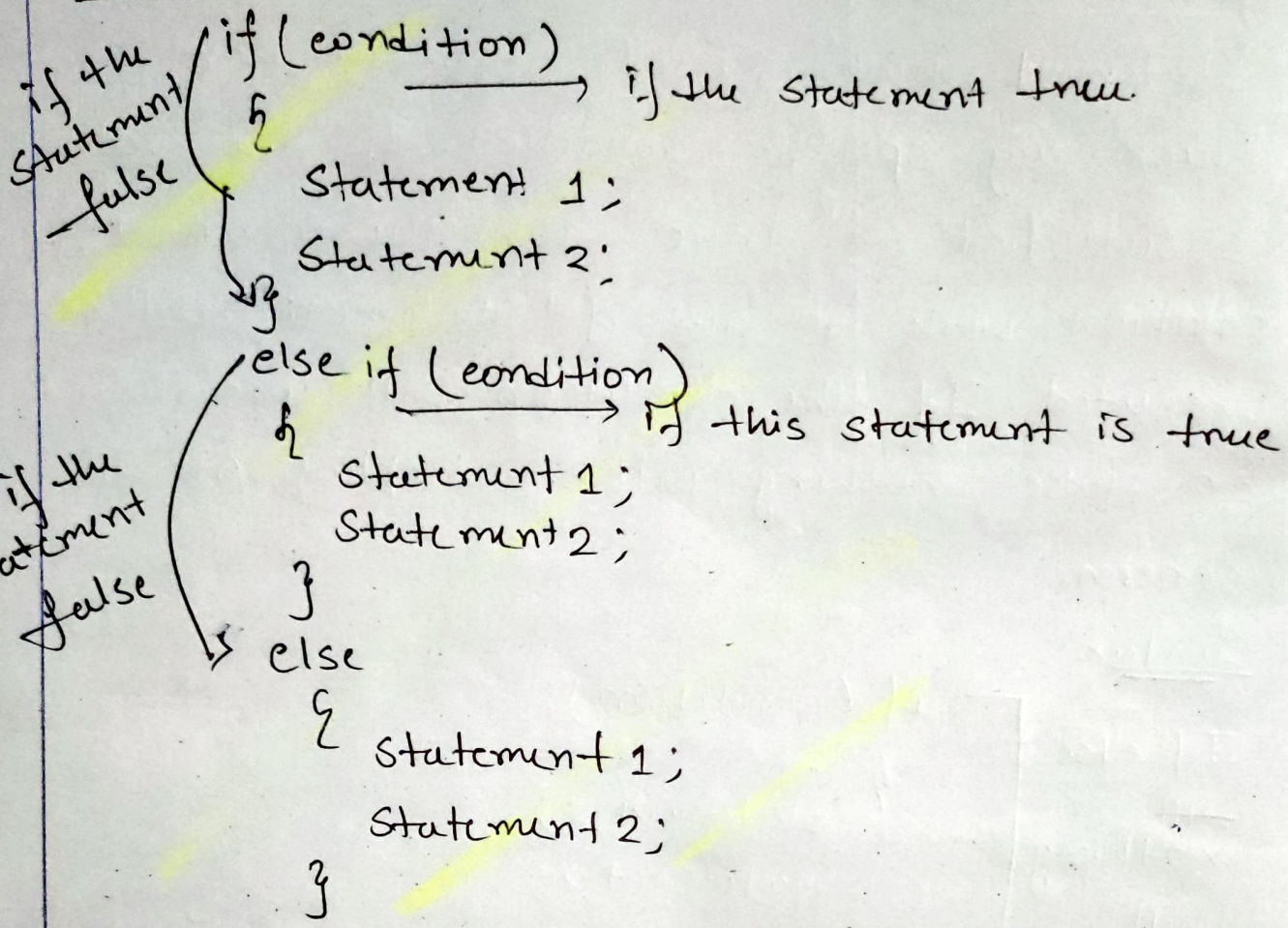
Syntax'r

If else will nothing will display

```
if (condition)
{
    Statement 1;
    Statement 2;
    Stament 3;
}
```

If else statement; It is also a decisition muking statement which is used when we want to chuck 2 conditions in our ~~pro~~ programme. If the first part of the if statement is true, then the remaining portion with the if statement will be executed and if the first part is false, then the else portion will be executed.

Syntax:-



Switch case:-

It is a decision statement which is alternate of if else statement, But in case of switch only equality operator (==) is used, no logical operator can be present within swicth case statement. In case of switch, case is presents in place of else if statement. 'Break' is statement which is used within switch case on-loop statement.

Break:- This is a statement through which we can forcefully comeout from the loop on switch case.

Syntax:-

```
Switch (value)
{
  case 1 :
        statement 1 ;
        break;
  case 2;
        Statement 2;
        break;
  default
}
```

Default:- Is the last statement of switch case statement which is alternate of else statement.

(=) assignment opanctor:- It is a assingment ~~opetarn~~ opetrater, which is used to store any value with in the vanicable.

(==) equality opanetor:- It is a equality operater, which is used to chuck/compune/ variable with a value is equal or not.

Logical operator

AND (&&):- It is a logical operater which is used within if else statement & loop statement This statement is used to chuck whether all the conditions true, then remaining portion with in the if statement will be executed otherwise next else or else if portion will be executed.

OR (||):- It is also a logical operator which is used to chuck anyone of the condition is true or not within the if statement anyone of the condition is true, then the remain portion within the if Statement will be executed. if none of the condition is true then the else portion will be executed.

Divide (/):- It is a divide operator which is used to find out the quotient of any number.

Modulus (%):- It is a modulus operator which is used to find out the remainder of any no.

Nested for loop :-

1. If, total row & columes are not same,
& 1st value is changing & lowest
& Last value is fixed & highest
then j=i , j<= highest value , j is incrementing
(i is always changing)

Ex →
```
1 2 3 4
2 3 4
3 4
4
```

2. If, total row & columes are not same,
& 1st value is changing & highest,
& Last value is fixed & lowest
then j=i , j> = lowest value, j is Decrementing,
(i is changing)

ex:-
```
4 3 2 1
3 2 1
2 1
1
```

3. If, total row & columes are not same,
& 1st value is fixed & lowest,
& Last value is changing & highest,
then j= lowest value (fixed), j<=i, j is incrementing.

ex:-
```
1 2 3 4
1 2 3
1 2
1
```

4. If, total row & columns are not same,
& 1st value is fixed & highest,
& Last value is changing & lowest,
than, j = 1st value (highest), j>=i, j is decrementing (i is changing)

Ex:— 4 3 2 1
4 3 2
4 3
4

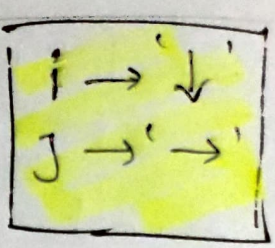
5. If, total no. of row & columns are same,
than there is no relationship between i & j
(row & columns) that means i & j ~~boths~~ both
loop will execute same no. of times j is
not depent on i.

Ex:— 1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4

6. If only a single value present with in the first
row than that most be the i's value and
j is depends on i.

Ex:- 1
1 2
1 2 3
1 2 3 4

Syntax:




```
for( :  ;  ){
    for( :  ;  ){
        {
        }
    }
}
```

outer for loop ← the outer `for` block

→ inner for loop (the inner `for` block)

Note:

- If with in the same row value not j's value, other wise print j's value.
- Inner for loop will execute more times than outer for loop.

Fuctions '()'

Q. What & why?

If we don't want to write a certain pices of code repeatedly to perform it multiple times, we can use functions, where we simple write that certain punt. and make the code more readable and easy to understand. as well as make it's complexity less.

why?

→ 1) to avoid Rapetation

[It is possible to use loops for do any action repetadly but not everything]

2) Make Readable.

ex:

```
#include<stdio.h>
int good(){
    printf("Egg is bowling\n");
return; } //return is end of func.
void main(){
    good();
}
```

output:-

Egg is bowling

I am calling the fuction good() to do its task.

if i write,

```
void main(){
    good();
    good();
    good();
}
```

output:-

Egg is bowling
Egg is bowling
Egg is bowling.

Basic syntax:—

```
int/void fun(){
    // code
    // code
}
main(){
    fun();
    // code
    // code
}
```

Code starts from main function then if there are extra function will execute later.

// Fuction of Function

```
#include<stdio.h>
int poland(){
    printf("This is poland");
}
int Dubai(){
    printf("This is dupai then");
    poland();   //calling poland();
}
int India(){
    printf("This is India. then");
    Dubai();    // calling dubai()
}
void main(){
    India();
}
// calling india()
```

output:-

This is India then
This is Dubai then
This is poland

Important point :-

1) main() comes only once.
2) program starts with main()
3) we can use unlimited function.

Return type :-

```
// Sum of two no.          -> Arguments / Parameters.
int sum (int a, int b) {
    return a+b;
}
void main() {
    int a=5, b=4, add;
    add = sum(a, b);
    printf("%d", add);
}
```

Return type ← int (name) sum

this variable and the arguments within the function declaration are NOT SAME

∴ sum(a,b) ≠ sum(int a, int b)

// sum(a, b) means, I am passing the values of variable a & b into the arguments of int a & int b of the declaration of sum() func.

sum(int a, int b)

- ☒ They are not same.
- ☑ They are copied

they (a,b) are copied into (int a, int b)

// Return type :- int

int return type means we have to return some integer.

Syntax :-

int func(a,b) {

Function prototype (kind of useless)

Normal code

```
#include<stdio.h>
int fun(){
    printf("hello");
}
void main(){
    fun();
}
```

with fun Prototype

```
#include<stdio.h>
void main(){
    int fun();
    fun();
}
int fun(){
    printf("hello");
}
```

```
#include<stdio.h>
int fun(); //declaration
void main(){
    fun(); //calling
}
int fun(){ //definition.
    printf("Hello.");
}
```

Scope of variable (limitation)

the variables which are declared in any function they can't work as any other function variable. that is known as the scope of that variable

ex:-

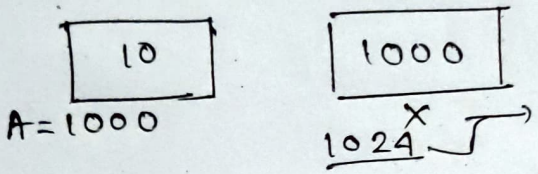
```
void fun(){
    int a,b; //this a & b only
    //code     accessable in fun()
}            function
void main(){
    int x,y; //this x & y only
    //code     accessable in 'main()
}            function.
```

Pointer

Q. What & why?

Pointers are kind of variables which are use to store any other variable's address. Not only storing the address we can also make changes of the values of other variables using pointer.

Let, an integer A=10 and its address 1000 and an int pointer x holding its address.



// It also take its own addrss btw.

Syntax:

```
int a=10;
int* x=&a;    // &a means address of a variable.
```

```
printf("%p", x);
```

output: 1000

// %p is use to print the value of that pointer type variable, which is the address of another variable

```
printf("%p", &x);
```

output: 1024

// &x will print the address of this pointer type variable 'x'

```
printf("%d", *x);
```

output: 10

// *x is use to print the value of that variable which address is holded by the pointer x.

pass by reference (with swap of two no.)

```
#include <stdio.h>
void swap(int * x, int* y){
    int temp;
    temp = *x; // temp = 2
    *x = *y; // *x mean value at a = 9 (which is value of b by *y)
    *y = temp; // *y mean value at b = 2
    return;
}
int main(){
    int a=2, b=9;
    swap(&a,&b); // &a and &b for pass the address
    printf("The value of a = %d ",a);
    printf("The value of b = %d ", b);
    return 0;
}
```

two pointer type arguments for store the address by a &a, &b

Recursion

Q. What & Why ?

When a fuction call itself for do any certain work or execute any certain part of a code is known as Recursion.

In case of Recursion we don't use loops as it's called it self and can do any work respectively according to the condition and terms.

// Mathematics,

If we need a factorial of any no., (Let 5)

$5! = 5 \times 4 \times 3 \times 2 \times 1$

or, $5! = 5 \times 4!$ now, $4! = 4 \times 3!$ OK, $3! = 3 \times 2!$

$\therefore \boxed{n! = n \times (n-1)!}$

$\therefore f(n) = n \times f(n-1)$

↳ Recurrence Relation

∴ we can see here a a func is calculated by using the same function.

// Now in C language,

```
# include <stdio.h>
int fact (int n) {
  if (n==1) return 1; // base case
  return n* fact (n-1) ; }
int main ( ) {
      int n= 5;
      printf ("%d", fact (n));
      return 0;
}
```

↳ the same func calling itself.

Trace Diagram :-

∴ 5!

⇒ 5 * 4!

⇒ 5 * 4 * 3!

⇒ 5 * 4 * 3 * 2!

∴ 5 * 4 * 3 * 2 * 1

= 120 (Ans)

fact(5)

↓

5 * fact(4) ⟶ 120

↓

4 * fact(3) ⟶ 24

↓

3 * fact(2) ⟶ 6

↓

2 * fact(1) ⟶ 2

↓

Base case return (1)

Function (Notes)

Function:- It is basically a data type which can be used to reuse the same thing or certain piece of code again & again without defining the something or rewirite the code multiple times.

Features:-

1) **Reuseability:-** It is a property within function which can be used to without write the same code again and again.

2) **Code-complexity:-** If we write any programme as a breakup of function then programming logic can be more preciese to the user or user can easily understand the code without much of complexity.

3) **Error Connection:-** By using function progra can be divided into smaller parts so in that case to check the error & rectify the error can be easier for the user.

4) **Time compexity:-** By using function user can rectify the error less time than the normal program so time complexity can be measured by using function.

Function

- System difined func / Inbuilt func
- user defind function

System defined Function :-

It is a function which can be created by the 'C' developer.

example:- printf( ), scanf( ), main( ), getch( )

User defined Function :-

It is a function which can be used by the user / can be created by the user.

Example:- add( ), multi( ), div( ), substruct( ),

Structure :-

i) Function declaration :- It must be declared before the main function.

ii) Function definition :- It always after the end of the main function that means